

2η Εργασία — Ασφάλεια Συστημάτων

Secure File Storage System — Αναφορά Υλοποίησης

Πανεπιστήμιο Ιωαννίνων — Τμήμα Πληροφορικής & Τηλεπικοινωνιών

1. Περιγραφή Υλοποίησης ανά Μέρος

A — Hash & Salt: hash_password()/verify_password() με PBKDF2-HMAC-SHA256 (390.000 επαναλήψεις, salt 16 bytes, κλειδί 32 bytes) μέσω cryptography. Salt ανά χρήστη + πολλαπλές επαναλήψεις δυσκολεύουν brute-force/rainbow table.

B — OTP & Εγγραφή: register_user() ελέγχει ύπαρξη, μη-χρήση και ορθότητα (case-insensitive) του OTP. Μετά την επιτυχή χρήση σημειώνεται used=True (μονόχρηστο), υπολογίζεται hash+salt και δημιουργούνται αυτόματα κλειδιά RSA.

Γ — Αυθεντικοποίηση: authenticate_user() ανακτά hash/salt και καλεί verify_password(). Σε κάθε αποτυχία (user δεν υπάρχει ή λάθος κωδικός) εμφανίζεται το ίδιο γενικό μήνυμα, ώστε να μην είναι δυνατή η απαρίθμηση έγκυρων usernames (user enumeration).

Δ — Κλειδιά RSA: Δόθηκε έτοιμο, χρησιμοποιήθηκε αυτούσιο: RSA-2048 ζεύγος αποθηκευμένο σε PEM στο keys/.

E — Anti-Replay / Nonce: is_nonce_valid() ελέγχει αν το nonce έχει ξαναχρησιμοποιηθεί (→ απόρριψη) ή είναι νέο (→ αποθήκευση με timestamp). Bonus: αυτόματος καθαρισμός nonces >24 ωρών.

ΣΤ — Κρυπτογράφηση & Υπογραφή: encrypt_file(): τυχαίο κλειδί 32B + nonce 12B, AES-256-GCM (authenticated encryption). sign_data(): RSA-PSS/SHA-256 πάνω στο (data+nonce), ώστε η υπογραφή να δεσμεύεται και στο συγκεκριμένο αίτημα, όχι μόνο στο περιεχόμενο.

Z — Upload / Download / CLI: upload_file(): read → nonce → sign → anti-replay check → encrypt → save (.enc + metadata .json σε base64). download_file(): αντίστροφη ροή + επαλήθευση υπογραφής πριν την τοπική αποθήκευση. Το CLI διατηρεί session (logged_in_user) — Upload/Download απαιτούν ενεργή σύνδεση.

2. Παράδειγμα Εκτέλεσης (πραγματικό transcript)

Πραγματική εκτέλεση: έκδοση OTP → αποτυχημένη εγγραφή (λάθος OTP) → επιτυχής εγγραφή → αποτυχημένη/επιτυχής σύνδεση → upload → download με επαλήθευση υπογραφής → replay attack.

```
>>> admin_issue_otp('bob')
[ADMIN] OTP για 'bob': 93AF0752
```

```
>>> register_user('bob', <λάθος OTP>, 'Str0ngP@ss')
[ΣΦΑΛΜΑ] Λανθασμένο OTP.
```

```
>>> register_user('bob', '93AF0752', 'Str0ngP@ss')
[OK] Ο χρήστης 'bob' εγγράφηκε επιτυχώς.
```

```
>>> authenticate_user('bob', 'WrongPassword')
[ΣΦΑΛΜΑ] Λάθος στοιχεία σύνδεσης.
```

```
>>> authenticate_user('bob', 'Str0ngP@ss')
[OK] Επιτυχής σύνδεση ως 'bob'.
```

```
>>> upload_file('bob', 'confidential.txt')
[OK] Το αρχείο 'confidential.txt' ανέβηκε και κρυπτογραφήθηκε επιτυχώς.
```

```
>>> download_file('bob', 'confidential.txt')
[OK] Η ψηφιακή υπογραφή του χρήστη 'bob' επαληθεύτηκε επιτυχώς.
[OK] Το αρχείο αποθηκεύτηκε ως 'downloaded_confidential.txt'.
```

```
>>> --- Προσομοίωση Replay Attack ---
>>> is_nonce_valid('940e24...f51') # 1η χρήση -> True
```

```
>>> is_nonce_valid('940e24...f51') # 2η χρήση (replay)
[ΠΡΟΕΙΔΟΠΟΙΗΣΗ] Εντοπίστηκε πιθανή Replay Attack — το nonce έχει ήδη χρησιμοποιηθεί!
```

Σημείωση: Επισυνάψτε επίσης πραγματικά screenshots από την εκτέλεση του CLI (python secure_storage.py): εγγραφή με OTP, σύνδεση, upload, download, απόρριψη replay.

3. Ανάλυση Μηχανισμών Ασφάλειας

Μηχανισμός	Τι προστατεύει	Πώς
PBKDF2 + Salt	Επιθέσεις brute-force / rainbow table στους κωδικούς	390.000 επαναλήψεις SHA-256 + μοναδικό salt ανά χρήστη
OTP μονής χρήσης	Μη εξουσιοδοτημένη εγγραφή λογαριασμών	Το OTP επισημαίνεται used=True μετά την πρώτη επιτυχή χρήση
Γενικό μήνυμα σφάλματος	User enumeration κατά τη σύνδεση	Το ίδιο μήνυμα για «δεν υπάρχει» και «λάθος κωδικός»
AES-256-GCM	Εμπιστευτικότητα + ακεραιότητα αρχείων	Authenticated encryption· νέο τυχαίο κλειδί/nonce ανά αρχείο
RSA-PSS + SHA-256	Μη-αποποίηση (non-repudiation) και ακεραιότητα υπογραφών	Υπογραφή με RSA-PSS/SHA-256 πάνω στο (data + nonce) με το ιδιωτικό κλειδί του χρήστη
Nonce / Anti-Replay	Replay attacks (επανάληψη παλιού έγκυρου αιτήματος)	Κάθε nonce καταγράφεται· επανάχρησή του απορρίπτεται

Nonce στην υπογραφή: αν η υπογραφή κάλυπτε μόνο το περιεχόμενο του αρχείου, ένας επιτιθέμενος θα μπορούσε να υποκλέψει ένα παλιό έγκυρο ζεύγος (δεδομένα+υπογραφή) και να το ξαναστείλει — η υπογραφή θα παρέμενε έγκυρη. Επειδή το nonce συμπεριλαμβάνεται στο υπογεγραμμένο payload, κάθε αίτημα δένεται με ένα μοναδικό, μονόχρηστο

αναγνωριστικό.

AEAD (AES-GCM) αντί απλού AES-CBC: το GCM παρέχει ταυτόχρονα εμπιστευτικότητα και ακεραιότητα.

Τροποποιώντας πειραματικά ένα byte ενός αποθηκευμένου .enc αρχείου, η `decrypt_file()` απέτυχε με σφάλμα επαλήθευσης, όπως αναμενόταν, αντί να επιστρέψει αθόρυβα λανθασμένα δεδομένα.

4. Συμπεράσματα

Η εργασία συνδύασε πρακτικά πολλαπλούς μηχανισμούς ασφάλειας σε ένα ενιαίο pipeline: ασφαλή αποθήκευση κωδικών (PBKDF2+salt), ελεγχόμενη εγγραφή μέσω μονόχρηστου OTP, συμμετρική κρυπτογράφηση με ακεραιότητα (AES-GCM), ασύμμετρη ψηφιακή υπογραφή για μη-αποποίηση (RSA-PSS), και προστασία από replay attacks μέσω nonce. Κάθε επίπεδο καλύπτει διαφορετική απειλή, δείχνοντας πώς σχεδιάζεται ένα ρεαλιστικό σύστημα ασφαλούς αποθήκευσης αρχείων.